```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

hr = pd.read_csv(r"C:\Users\Harikesh\Downloads\HRDataset_v14.csv")

hr.head()

hr.info()

hr.isnull().sum()

hr.describe()

hr.drop(columns='DateofTermination', inplace=True)

 # FEATURE ENGINEERING

hr.columns = hr.columns.str.strip()

# Create Age from DOB

hr['DOB'] = pd.to_datetime(hr['DOB'])

# Calculate Age

current_year = pd.Timestamp.now().year
hr['Age'] = current_year - hr['DOB'].dt.year

# Create age Groups
hr['Age_Group'] = pd.cut(
    hr['Age'],
    bins = [-49,30,40,50,],
    labels=[
        'Young Employees',
        'Mid Level Employees',
        'Senior Employees',
       #'Experience Employees'
    ],

    include_lowest=True
)

hr[['DOB','Age','Age_Group']].head()

hr['Age_Group'].unique()

import pandas as pd

# Convert date columns into datetime format
# We use pd.to_datetime() to convert the column into a proper date format so Python can perform date calculations correctly.

#In your dataset, DateofHire is usually stored as text/string, not as an actual date.
hr['DateofHire'] = pd.to_datetime(hr['DateofHire'])

# Calculate Total Years in Company
# it gives the current date and time pd.Timestamp.now().
current_date = pd.Timestamp.now()
```

```python


# pd.cut() is used to divide continuous numeric data into groups or categories (bins).

# In your project, it is used to convert employee experience years into meaningful categories.
hr['Total_Years_in_Company'] = (
    (current_date - hr['DateofHire']).dt.days / 365
).round(1)

# Create Tenure Categories
hr['Tenure_Category'] = pd.cut(
    hr['Total_Years_in_Company'],
    bins=[0, 1, 3, 5, 100],
    labels=[
        'New Joiner',
        '1-3 Years',
        '3-5 Years',
        '5+ Years'
    ],
    include_lowest=True
)

# Display result
hr[['DateofHire', 'Total_Years_in_Company', 'Tenure_Category']].head()

# Marital Status Group

hr['MaritalStatusID'].value_counts()

hr[['MaritalStatusID','MaritalDesc']].drop_duplicates()

hr['Marital_Status_Group'] = hr['MaritalStatusID'].replace({
    0: 'Single',
    1: 'Married',
    2: 'Divorced',
    3: 'Divorced',
    4: 'Divorced'
})

hr['Marital_Status_Group'].value_counts()

hr['PerformanceScore'].value_counts()

# This Methode are used to combine or create group same of  elemenet
hr['Performance_Group'] = hr['PerformanceScore'].replace({
    'Exceeds' : 'High Performance',
    'Fully' : 'Good Performance',
     'Needs Improvement' : 'Low Performance',
     'PIP' : 'Low Performance'
})

hr[['PerformanceScore','Performance_Group']]

hr['Performance_Group'].value_counts()

hr['EngagementSurvey'].value_counts()

hr['Engagement_Bands'] = pd.cut(
```

```
    hr['EngagementSurvey'],
    bins=[0,2,4,5],
    labels=[
        'Low Engagement',
        'Medium Engagement',
        'High Engagement'
    ],
    include_lowest= True
)
hr[['Engagement_Bands','EngagementSurvey']]

hr['Engagement_Bands'].values.value_counts()

hr['Group_Absence'] = pd.cut(
    hr['Absences'],
    bins = [0,5,10,17],
    labels=[
        'Low Absence',
        'Moderate Absenece',
        'High Absence'
    ],
    include_lowest=True
)

hr[['Group_Absence','Absences']]

hr['Group_Absence'].value_counts()

hr['DaysLateLast30'].value_counts()

hr['Late_Arrival_Risk'] = pd.cut(
    hr['DaysLateLast30'],
    bins=[-1,0,2,10],
    labels=[
        'Never Late',
        'Occasionally Late',
        'Frequently Late'
    ]
)

hr[['DaysLateLast30','Late_Arrival_Risk']]

hr['Late_Arrival_Risk'].value_counts()

# Employment Features

hr['Termd'].value_counts()

hr['Employment_Group'] = hr['EmploymentStatus'].replace({
    'Active' : 'Active',
    'Voluntarily Terminated' : 'Left Company',
    'Terminated for Cause' : 'Left Company'
})

hr[['Employment_Group','EmploymentStatus']].head()

hr['Department_Group'] = hr['Department'].replace({
    'Production' : 'Sales & Marketing',
    'IT/IS' : 'Technical',
    'Software Engineering' : 'Technical',
```

```
    'Admin Offices' : 'Operations',
    'Executive Office' : 'Operations'
})

hr[['Department_Group','Department']].head()

# Salary Engineering

hr['Salary'].value_counts()

hr['Salary'].describe()

hr['Salary_Band'] = pd.cut(
    hr['Salary'],
      bins=[0, 50000, 80000, 220450],
    labels=[
        'Low Salary',
        'Medium Salary',
        'High Salary'
    ],
    include_lowest=True
)

hr[['Salary','Salary_Band']]

# Hiring(Year,Month,Day)

# Convert DateofHire into Datetime format
hr['DateofHire'] = pd.to_datetime(hr['DateofHire'])


# Extract Hiring Year

hr['Hiring_Years'] = hr['DateofHire'].dt.year

# Extract Hiring Month

hr['Hiring_Month'] = hr['DateofHire'].dt.month

# Extract Hiring month name
hr['Hiring_Month_Name'] = hr['DateofHire'].dt.month_name()

# Extract Hiring Quarter

hr['Hiring_Quarter'] = hr['DateofHire'].dt.quarter

hr[[
    'DateofHire',
    'Hiring_Years',
    'Hiring_Month',
    'Hiring_Month_Name',
    'Hiring_Quarter'
]]

df = hr.drop(columns=[
    'Employee_Name',
    'EmpID',
    'Zip',
    'ManagerName'
])
```

```

## # EDA and Visualization

# Employee Attrition Analysis

import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x = 'Employment_Group',data = hr)

plt.title('Employee Attrition Analysis')
plt.xlabel('Employment Status')
plt.ylabel('Number of Employees')

# Rotate labels for better visibility
plt.xticks(rotation = 15)

plt.show()


# Heatmap

# They are only selected numerical columns

numeric_df = hr.select_dtypes(include=['number'])
corr = numeric_df.corr()

plt.figure(figsize=(12,8))
sns.heatmap(corr,
            annot=True,
            fmt='.2f',
            linewidths=0.5)


# Salary Band

sns.countplot( x = 'Salary_Band', data=hr)
plt.title('Employee Salary')
plt.xlabel('Salary')
plt.ylabel('Number of Employee')

plt.show()

# Salary Distribution

sns.histplot(hr['Salary'], bins=20)

plt.title('Salary Distribution')
plt.xlabel('Salary')
plt.ylabel('Frequency')

plt.show()

# Salary Distribution

sns.boxplot(
    x = 'Employment_Group',
    y = 'Salary',
    data = hr
```

```
)

plt.title('Salary by Employment Status')
plt.show()

# Performance Analysis

sns.countplot(x = 'Performance_Group',data=hr)

plt.title('Performance Analysis')
plt.xlabel('Performance')
plt.ylabel('Number of Employee')

plt.show()

# Employee Satisfaction Analysis

sns.histplot(hr['EmpSatisfaction'], bins=5,kde=True)

plt.title('Employee Satisfaction Distribution')
plt.xlabel('Satisfaction Score')
plt.ylabel('Frequency')

plt.show()

# Absence Analysis

sns.boxplot(
    x = 'Employment_Group',
    y = 'Absences',
    data = hr
)


plt.title('Absences by Employment Status')
plt.show()

# Late Arrival Analysis

sns.countplot(x = 'Late_Arrival_Risk',data = hr)

plt.title('Late Arrival Analysis')
plt.xlabel('Arrival')

plt.ylabel('Number of Empoyees')

plt.show()

# Hiring Trend Analysis

sns.countplot(x='Hiring_Years', data=hr)

plt.title('Hiring Trend')
plt.xlabel('Year')
plt.ylabel('Number of Employees')

plt.xticks(rotation=45)

plt.show()

```

```python
# Department-wise Salary Analysis

sns.boxplot(
    x = 'Department',
    y = 'Salary',
    data = hr
    )

plt.title('Department wise Salary Analysis')
plt.xlabel('Department')
plt.ylabel('Salary')

plt.xticks(rotation = 45)

plt.show()

# Age Distribution

plt.figure(figsize=(12,8))

# Age Distribution
sns.countplot(x='Age_Group', data=hr)

# Title and labels
plt.title('Age Distribution')
plt.xlabel('Age Group')
plt.ylabel('Number of Employees')

plt.show()

# Machine Learning to Predict Attrition

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load Dataset
df = pd.read_csv(r"C:\Users\Harikesh\Downloads\HRDataset_v14.csv")

# Fill Missing Values
df = df.fillna(0)

# Convert String Columns
le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col].astype(str))

# Target Column
y = df['Termd']

# Features
X = df.drop(columns=['Termd'])

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
417
418 # Scaling
419 scaler = StandardScaler()
420
421 X_train = scaler.fit_transform(X_train)
422 X_test = scaler.transform(X_test)
423
424 # Logistic Regression Model
425 model = LogisticRegression(max_iter=1000)
426
427 model.fit(X_train, y_train)
428
429 # Prediction
430 y_pred = model.predict(X_test)
431
432 # Accuracy
433 print("Accuracy :", accuracy_score(y_test, y_pred) * 100)
434
435
```